宁波大学信息学院

实验报告

实验课程名称： 汇编语言与接口设计

学　　院： 信息学院

专　　业： 计算机科学与技术

指 导 教 师： 陈瑜

报告人姓名： 顾凯琪 **学号：** 226001624

班　　级： 22 级计算机科学与技术 2 班

学　　期： 2024-2025 第一学期

信息学院实验中心制

实验								
成绩								

实验项目名称	实验 8　DOS、BIOS 调用

一、　　实验目的与要求：

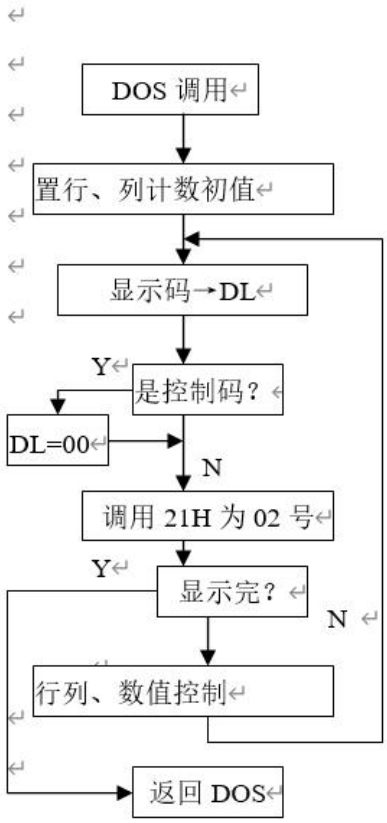
编一程序，将这些 ASCII 码表示的字符以 16 行 16 列的表格形式显示出来，要求显示时按 ASCII 码的递增顺序分行显示(即行内 ASCII 码依次递增,与 ASCII 表中给出的表格行、列位置正好倒置)。每行、相邻的两字符间由空白符（ASCII 码 00）或空格（ASCII 码 20H）分开，只要求显示出字符，不要画出表格线。

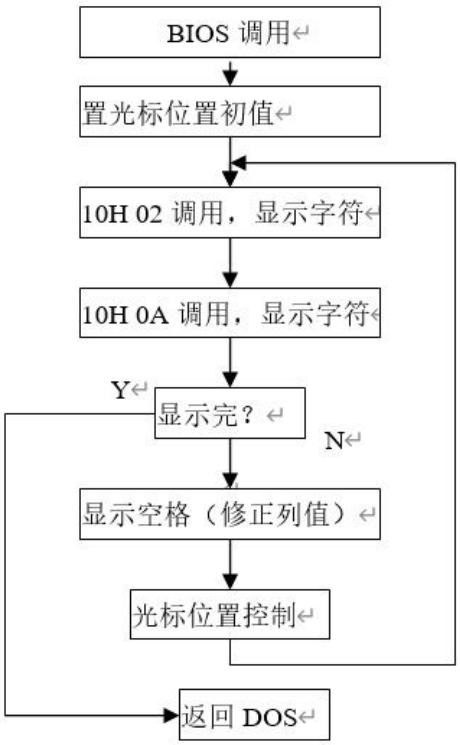
二、　　实验设备：

Masm for windows 集成实验环境共享版

Windows11 OS

三、　　实验方法（原理、流程图）





四、实验过程、步骤及内容

实验代码如下：

BIOS

```
1. DATA SEGMENT
2.
3. DATA ENDS
4. STACK SEGMENT STACK 'STACK'
5.  STA DB 128 DUP(?)
6.  TOP EQU LENGTH STA
7. STACK ENDS
```

此处声明了堆栈段，STA 用于定义一个 **128** 字节的堆栈空间，TOP 是堆栈顶指针的常量，用于设置 SP（堆栈指针）。堆栈用于保存临时数据和控制结构的返回地址。

```
8. CODE SEGMENT
9. ASSUME CS:CODE,DS:DATA,SS:STACK
10.START:
11. ;初始化相关寄存器
12.  MOV AX,DATA
13. MOV DS,AX
14. MOV AX,STACK
15. MOV SS,AX
16. MOV SP,TOP
```

程序在此开始，并通过 **ASSUME** 指令告诉编译器各段寄存器的对应段名。

MOV DS, AX：将 **DATA** 段地址加载到 **DS** 寄存器。

MOV SS, AX 和 **MOV SP, TOP**：将 **STACK** 段地址加载到 **SS**，并初始化堆栈指针 **SP**。

```
17.
18. MOV BL,0;BL 中存放要打印字符的 ASCII 码
19. MOV DH,0;设置行号
```

BL 存储当前的 **ASCII** 码字符（从 **0** 开始），DH 表示行号。外循环和内循环将用于逐行逐列输出字符。

```
20.OUTER:
21. MOV DL,0;设置列号
22.INNER:
23. ;输出 ASCLL 表的一个字符
24. MOV AH,02H ;BH=0 显示页码,DH=行(Y 坐标),DL= 列(X 坐标)
25. INT 10H;调用 BIOS 中断设置光标位置
```

外循环和内循环分别控制行号和列号，每次循环根据 DH 和 DL 设置字符的显示位置。

MOV AH, 02H 和 INT 10H 使用 BIOS 中断 10H 设置光标到 (DH, DL) 位置。

```
26. CMP BL,0AH;判断是否为换行符
27. JZ CHANGE
28. CMP BL,0DH;判断是否为回车符
29. JZ CHANGE
30. MOV AL,BL;将 BL->AL 用于显示
31.SHOW:
32. ;输出字符
33. MOV CL,1;设置输出字符字数
34. MOV AH,0AH ;AL=字符，BH=页码，CX=多次打印字符
35. INT 10H;调用 BIOS 中断输出字符
```

通过 CMP BL, 0AH 和 CMP BL, 0DH 检查当前字符是否是特殊字符换行或回车。如果是，则跳转到 CHANGE 标签，将字符更改为 0 进行替代显示。如果不是特殊字符，MOV AL, BL 将 ASCII 码存入 AL，然后调用中断 10H 显示字符。

```
36. ;输出间隔的空格
37. INC DL;列号+1
38. INC BL;下一个字符 ASCII 码
39. MOV AH,02H;设置光标位置
40. INT 10H
41. MOV AL,0
42. MOV AH,0AH
43. INT 10H;输出空格间隔
```

显示字符后，DL 列号自增，BL ASCII 码递增，准备显示下一个字符。随后再次调用中断显示空格，形成字符间的间隔。

```
;输出 32 个为一行，总共 16 行
INC DL;列号+1
CMP DL,32;判断一列是否完成
JNZ INNER
INC DH
CMP DH,16;判断行数是否达标
JNZ OUTER
```

当列数达到 32 时，内循环跳出，开始显示下一行字符，直到达到 16 行。

```
MOV AX,4C00H
INT 21H;结束程序
```

当 16 行全部显示完毕，程序调用中断 21H 以 4C00H 退出。

```
CHANGE:;若为特殊字符则将其替换为 0
        MOV AL,0
JMP SHOW

CODE ENDS
END START
```

DOS

```
DATA SEGMENT
DATA ENDS
STACK SEGMENT STACK 'STACK'
 STA DB 128 DUP(?)
 TOP EQU LENGTH STA
STACK ENDS
```

这里定义了堆栈段 STACK，其中 STA 分配了 128 字节的堆栈空间，而 TOP 作为堆栈的顶端位置，用于初始化堆栈指针。

```
CODE SEGMENT
ASSUME CS:CODE,DS:DATA,SS:STACK
START:
;初始化相关寄存器
   MOV AX,DATA
MOV DS,AX
```

```
MOV AX,STACK
MOV SS,AX
MOV SP,TOP
```

通过 ASSUME 语句指定各段寄存器的使用。

使用 MOV 指令将数据段地址装入 DS，堆栈段地址装入 SS，并初始化堆栈指针 SP。

```

;置行、列计数初值,字符 0 开始
MOV BL,0;BL 中存放要打印字符的 ASCII 码
MOV CX,16;设置外层循环次数，即行数
```

此处初始化 BL 寄存器为 0（表示第一个要打印的字符的 ASCII 码），并将 CX 设为 16 以控制行数。

```
OUTER:
 PUSH CX
 MOV CX,16;设置内层循环次数，即列数
INNER:
 CMP BL,0AH;判断是否为换行符
 JZ CHANGE
 CMP BL,0DH;判断是否为回车符
 JZ CHANGE
 MOV DL,BL;显示码→DL
```

外层循环控制行数，每行有 16 列。

PUSH CX 和 POP CX 保护外层循环的计数器。

CMP BL, 0AH 和 CMP BL, 0DH 判断当前字符是否是控制字符（换行符和回车符）。如果是，跳转到 CHANGE 段，将其替换为 0 显示。

```
SHOW:
 ;输出 DL 里的内容
 MOV AH,2
 INT 21H ;int 21 02 号功能，输出值放 DL

 ;输出空格间隔
 MOV DL,0
 INT 21H;输出空格间隔
```

```
37. INC BL;下一个字符 ASCII 码
38. LOOP INNER
```

在 SHOW 段，调用中断 21H 的 02H 功能输出 DL 中的字符。

接着通过设置 DL 为 0，再次调用 21H 中断输出空格，作为字符间隔。

INC BL 将 ASCII 码自增 1，显示下一个字符。

```
39. CALL CRLF;换行函数
40. POP CX
41. LOOP OUTER
42.
43. MOV AX,4C00H;退出程序
44. INT 21H
```

在每行输出完成后，调用 CRLF 子程序换行。

POP CX 恢复外层循环计数器。

当所有行输出完毕，调用 21H 的 4C00H 功能正常结束程序。

```
45.CHANGE:;是控制码则将其替换为 0
46. MOV DL,0
47. JMP SHOW
```

如果遇到换行或回车符，通过 CHANGE 将 DL 置为 0 显示一个空字符。

```
48.;换行函数
49.CRLF    PROC    NEAR
50. MOV DL,0DH
51.    MOV AH,02H
52.    INT 21H        ;光标移到第一列
53.    MOV DL,0AH
54.    MOV AH,02H
55.    INT 21H        ;光标移到下一行
56.    RET
57.CRLF ENDP
58.CODE ENDS
59.END START
```

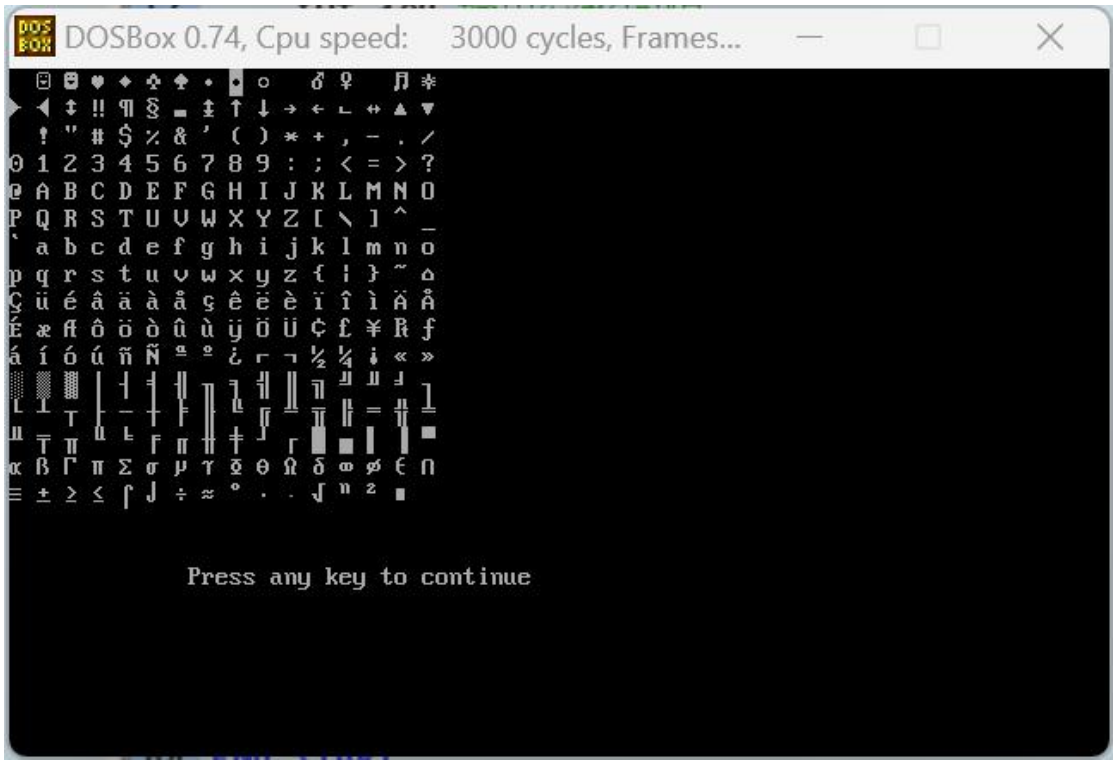
CRLF 子程序用于换行：

MOV DL, 0DH 将 DL 设为回车符，并调用中断 21H 的 02H 功能，将光标移至行首。

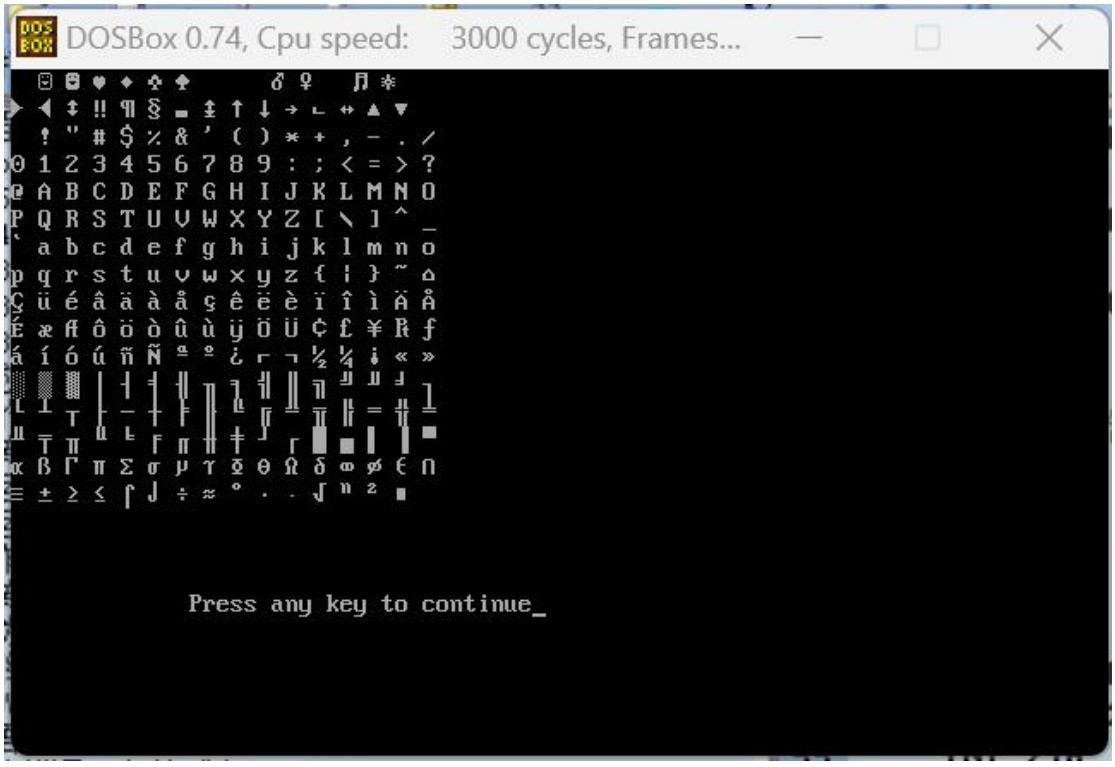
MOV DL, 0AH 设置为换行符，再次调用中断 **21H**，使光标移至下一行。

五、实验数据（现象）处理分析

BIOS 结果


DOSBox 0.74, Cpu speed: 3000 cycles, Frames...
Press any key to continue

DOS 结果


DOSBox 0.74, Cpu speed: 3000 cycles, Frames...
Press any key to continue_

六、实验结论：

本实验通过汇编语言编写了一个 **ASCII** 字符输出程序，验证了如何利用 **BIOS** 和 **DOS** 中断 **21H** 显示字符以及控制光标位置，实现了 **16** 行、每行 **16** 个字符的排布输出。通过使用循环控制行列数，并使用条件判断处理换行和回车符，实验展现了在汇编语言中处理控制字符及实现基本字符显示的原理和方法。这次实验加深了对汇编语言中循环控制、条件判断和中断调用的理解。

DOS 和 **BIOS** 中断调用各有优越性，适用于不同场景：

DOS 调用的优越性：**DOS** 中断（如中断 **21H**）提供了高层次的操作，抽象了硬件细节。它简化了文件操作、字符显示和内存管理等功能的实现，使程序员可以通过简洁的调用指令完成复杂的操作，代码编写更简单、通用性更强，特别适合需要兼容不同硬件的应用程序。

BIOS 调用的优越性：**BIOS** 中断直接访问硬件层面，提供了对键盘、显示器等设备的低级控制，反应速度更快，更适合对速度要求较高的系统级程序。**BIOS** 调用依赖硬件，执行效率高，适用于需要直接控制硬件、实现精细操作的场景，但代码的硬件依赖性较高，不同硬件平台兼容性较差。

总结来说，**DOS** 调用便于应用层开发，而 **BIOS** 调用适合底层硬件控制。

七、指导教师批阅意见：

成绩评定：

指导教师签字：

年　　月　　日

八、备注：

注：报告内的项目或内容设置，根据实际情况可加以调整和补充，写不下可另附一页。